PREJUD · RELATÓRIO TÉCNICO DA SESSÃO

Fluxo público de aceite/contestação, regras Firestore e trilha de integridade

Documento consolidado da sessão com tudo que foi executado, problemas encontrados, estado atual do projeto, ponto exato de parada e continuidade recomendada em conformidade com o manual técnico do PreJud.

MÓDULO PRINCIPAL

Acordo público /p/[protocol]

TEMA DA SESSÃO

Confirmação, contestação, regras e hash

ESTADO FINAL

Parou com page.tsx corrompido e sem backup

1. Resumo executivo

O que funcionou: o teste com regras ultra-permissivas comprovou que os botões de *Confirmar* e *Contestar* conseguem alterar status no fluxo público. Isso demonstrou que o fluxo base do cliente e a atualização do agreement funcionam quando a barreira é removida.

O que não ficou estável: a tentativa de reintroduzir segurança nas regras do Firestore e, depois, de adicionar visualização do hash SHA-256, gerou retrabalho, mudanças parciais e por fim um erro de sintaxe em `app/p/[protocol]/page.tsx`.

Ponto crítico atual: o arquivo da página pública ficou corrompido por substituições automáticas com regex e o terminal registrou que **não foi encontrado backup** da página. Antes de seguir com novas features, a prioridade é restaurar a página pública para um estado compilável.

2. O que foi feito nesta sessão

ETAPA	AÇÃO EXECUTADA	RESULTADO
Diagnóstico inicial	Deploy de regras ultra-permissivas e teste do fluxo público	**Concluído** Confirmou que os botões funcionam sem bloqueio de regra
Validação funcional	Teste visual do botão confirmar e do botão contestar	**Concluído** Houve mudança correta de status e eventos
Restauração de segurança	Restauração do backup de `firestore.rules` e tentativas de correção	**Parcial** Várias tentativas; houve problemas de interpolação e sintaxe
Regras Firestore	Ajustes em `hasValidClientToken()`, `isValidPublicUpdate()` e leitura de `agreement_events`	**Parcial** Deploy foi concluído em parte, mas a estratégia de leitura pública ficou inconsistente
Serviço principal	Alteração de `processPublicAgreementConfirmation` para aceitar token	**Aplicado** Token passou a ser enviado nas ações de aceite/contestação
Página pública	Alteração para passar token na carga inicial e nas ações	**Aplicado** Carga inicial e botões passaram a incluir o token
UX dos botões	Separação de estado de loading para aceite e contestação	**Corrigido** O problema de “ambos girando ao mesmo tempo” foi tratado
Integridade	Inclusão de campo `hash`, geração e validação básica	**Parcial** Implementado, mas com algoritmo inadequado e visualização quebrada
Interface	Tentativa de exibir o hash na página pública	**Quebrou** Introduziu JSX inválido em `page.tsx`

3. Diagnóstico técnico consolidado

3.1 O que o teste ultra-permissivo provou

- O código de ação pública consegue processar a mudança de status quando a regra não bloqueia.

- O problema central não era o clique do botão em si.
- O gargalo estava no desenho entre regras Firestore, leitura pública do agreement, leitura da timeline e transporte do token.

3.2 Principais problemas encontrados

PROBLEMA	CAUSA OBSERVADA	IMPACTO
PowerShell interpolando `$(database)` e `$(request.auth.uid)`	Uso de here-string inadequado para conteúdo literal do Firestore Rules	Quebra de comandos e perda de tempo em deploy
Leitura pública falhando	Regra dependia de token em contexto que o SDK do Firestore não injeta como query arbitrária	Página pública com “Missing or insufficient permissions”
Timeline pública problemática	`getAgreementEvents()` participava da leitura inicial e herdava a restrição	Mesmo com agreement lido, eventos podiam bloquear a página
Hash chamado de SHA-256 sem ser SHA-256 real	Foi criado um hash numérico simplificado, não criptográfico	Viola o objetivo de trilha verificável e integridade séria
Arquivo público corrompido	Substituições regex sucessivas em JSX	Aplicação passou a retornar erro 500 de compilação

4. Estado atual real do projeto ao final da sessão

Itens que ficaram aproveitáveis

- Separação do loading dos botões.
- Passagem de token para a ação pública.
- Passagem de token na carga inicial do agreement.
- Confirmação prática de que o fluxo público funciona quando a regra permite.

Itens que exigem revisão imediata

- `app/p/[protocol]/page.tsx` com JSX inválido.
- Implementação do hash anunciada como SHA-256, mas incorreta.
- Estratégia de leitura pública em `firestore.rules` provavelmente aberta demais.
- Ausência de backup localizado para restaurar a página pública.

5. Onde exatamente parou

O trabalho parou no momento em que a aplicação passou a falhar na compilação da rota pública `/p/[protocol]` por erro de JSX em `frontend/app/p/[protocol]/page.tsx`, após várias tentativas de inserir o bloco visual do hash.

Último estado observável: o terminal mostrava erro de parsing com *Expected ', apontando para a região em que o bloco do hash foi inserido de forma quebrada.*

*Além disso, foi informado no terminal: **"SEM BKP ENCONTRADO"**, o que significa que a continuidade precisa começar por reconstrução segura do arquivo ou recuperação por histórico local, Git, undo stack, ou cópia de outro ambiente.*

6. O que falta fazer

PRIORIDADE	TAREFA PENDENTE	OBJETIVO
Alta	Restaurar `app/p/[protocol]/page.tsx` para estado compilável	Voltar a rodar a aplicação sem erro 500
Alta	Validar se as correções úteis de token e loading permaneceram no arquivo restaurado	Não perder os ajustes bons da sessão
Alta	Revisar `firestore.rules` com estratégia segura de leitura pública	Evitar leitura totalmente aberta de agreements
Média	Refazer a trilha de hash com SHA-256 real	Atender o manual de trilha verificável
Média	Exibir hash e protocolo de forma estável na UI	Dar transparência ao cliente antes do aceite
Média	Testar novamente aceite e contestação com regra final segura	Fechar o ciclo sem usar regra ultra-permissiva

7. Continuidade exata a partir do ponto de parada

Seguindo o manual, a continuidade deve respeitar a ordem **Domínio** → **Permissões** → **State Machine** → **Timeline** → **UI**. Nesta sessão, a UI foi alterada repetidas vezes antes de estabilizar totalmente domínio e permissões. Portanto, a retomada correta é:

1. **Recuperar a UI pública** para um estado compilável, sem tentar adicionar mais nada.
2. **Conferir domínio e permissões** antes de voltar a mexer em hash e visual.
3. **Somente depois** recolocar a exibição do hash.

Próxima ação recomendada no terminal

> A próxima ação não deve ser inventar mais regex. O primeiro passo é abrir o arquivo quebrado, localizar o bloco corrompido do hash e reconstruir manualmente a seção do cabeçalho da página pública.

```
Get-Content -LiteralPath "C:\prejud-saas-new\frontend\app\p\[protocol]\page.tsx" -
TotalCount 220
```

Depois de inspecionar o arquivo inteiro, a reconstrução segura recomendada é restaurar manualmente somente a região do cabeçalho onde aparecem protocolo, hash e os botões,

preservando as correções úteis já feitas: token na leitura inicial, token nas ações e estados de loading separados.

Bloco lógico que deve existir após a restauração

```
// estados de loading separados
const [confirmLoading, setConfirmLoading] = useState(false);
const [contestLoading, setContestLoading] = useState(false);

// carga inicial com token
const data = await getAgreementByProtocolOrId(protocol, token || undefined);

// ações com token
await processPublicAgreementConfirmation(agreement.id, "accept", token || undefined);
await processPublicAgreementConfirmation(agreement.id, "reject", token || undefined);

// exibição visual segura do hash
{agreement.hash ? (
  <div className="mt-2 flex items-center gap-2 text-xs text-gray-500 bg-gray-900/50 p-2
rounded">
    <Shield className="h-3 w-3 text-emerald-500" />
    <span className="font-mono truncate" title={agreement.hash}>
      Hash: {agreement.hash.substring(0, 20)}...
    </span>
  </div>
) : (
  <div className="mt-2 text-xs text-gray-600 bg-gray-900/30 p-2 rounded">
    Hash: N/A (legado)
  </div>
)}
```

8. Regras do manual que precisam ser respeitadas daqui para frente

REGRA DO MANUAL	APLICAÇÃO PRÁTICA NESTA CONTINUIDADE
Agreement é a entidade central	*Toda solução deve partir de `types/agreement.ts` e do contrato real de estados e eventos*
Fluxo obrigatório: Domínio → Permissões → State Machine → Timeline → UI	*Não voltar a mexer pesado na tela antes de estabilizar regras e machine*
Nunca ignorar a state machine	*Confirmar e contestar devem respeitar os estados válidos de transição*
Nunca bypassar timeline para registrar eventos	*A atualização do agreement deve continuar gerando evento cronológico rastreável*
Não criar lógica de domínio em componentes	*Hash, token, validação e transição devem permanecer no serviço/domínio, não na tela*

9. Observação importante sobre o hash

> *O que foi implementado nesta sessão **não é SHA-256 real**. Foi criado um hash simplificado baseado em iteração numérica. Isso pode servir como marcador temporário, mas **não atende** ao conceito de "registro imutável" nem à trilha verificável descrita para o produto.*

Portanto, ao retomar este item, o correto é substituir essa lógica por geração criptográfica real com API adequada do ambiente, e então registrar esse valor de forma coerente no agreement e, idealmente, também nos eventos relevantes da timeline.

10. Conclusão operacional

A sessão foi útil porque isolou a causa principal do fluxo público e confirmou o comportamento esperado dos botões. Porém, o fechamento da sessão deixou a aplicação em estado instável por causa da quebra em `page.tsx` e da implementação apressada do hash.

> ***Continuidade correta:** recuperar a página pública, preservar as correções boas, estabilizar permissões conforme o manual e só então reintroduzir o hash de maneira séria e auditável.*

Relatório preparado para exportação em PDF. Recomendado salvar/imprimir em “A4” com margens padrão do navegador.